

Common Part 3: Combinatorial Optimization

Common Part 3: KO

Combinatorial optimization problems: formulation, complexity analysis, algorithms, and example applications.

Primary source repositories consulted: mohwald/oi-mszz, pan-sveta/oi-si-statnice, Pryx/oi-si-mszz, dralii/oi-mszz. The final text below is synthesized and corrected against standard combinatorial optimization material.

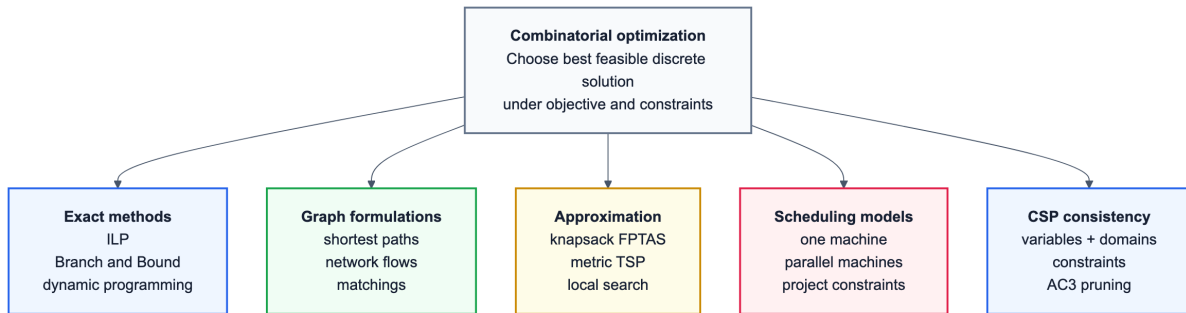


Figure 1: Combinatorial-optimization method map: exact methods, graph formulations, approximation, scheduling, and CSP consistency.

KO in Plain English

KO is about turning discrete choices into optimization problems. For each topic, know the model, whether it is easy or hard, and the standard exact, approximate, or graph algorithm.

1. Model the decision.

Variables, constraints, objective.

2. Pick the method.

Exact, polynomial graph algorithm, approximation, or DP.

3. State the guarantee.

Optimal, feasible, pseudo-polynomial, or approximate.

ILP and Branch & Bound

Use variables to describe choices.

- LP is polynomial; general ILP is NP-hard.
- Binary variables are yes/no switches.
- Branch & Bound splits cases and prunes by bounds.

Shortest paths and flows

Polynomial graph problems with strong structure.

- Dijkstra: nonnegative edges.
- Bellman-Ford: negative edges, detects negative cycles.
- Max-flow min-cut links augmenting paths to cuts.

Knapsack and TSP

Hard problems where DP and approximation matter.

- Knapsack DP is pseudo-polynomial.
- FPTAS scales profits for approximation.
- Metric TSP has double-tree and Christofides.

Scheduling

Assign jobs to time/resources.

- Use Graham notation to describe the problem.
- Know Bratley, Horn, list scheduling, McNaughton.
- ILP models use start/order/time-indexed variables.

CSP and AC3

Reduce variable domains by consistency.

- CSP = variables, domains, constraints.
- Arc consistency removes unsupported values.
- AC3 repeats revisions until stable or empty domain.

How to answer

Start with the model; then discuss algorithmic status.

- What are the variables?
- What is optimized?
- Is the method exact, approximate, or heuristic?

Exam sentence: ILP is powerful because it models discrete choices with linear constraints, but integrality makes it NP-hard; Branch & Bound solves it by relaxing, branching, and pruning.

Three traps

- Pseudo-polynomial does not mean polynomial in input length.
- Dijkstra needs nonnegative edge weights.
- Approximation ratios depend on minimization vs. maximization.

Use this page to choose the right method before reading formulas.

1. Integer Linear Programming and Branch and Bound

Linear and integer linear programs

A linear program (LP) optimizes a linear objective under linear constraints:

$$\min c^T x \quad \text{s.t.} \quad Ax \leq b, \quad x \geq 0.$$

An integer linear program (ILP) additionally requires some or all variables to be integer:

$$x_i \in \mathbb{Z}$$

or, in binary ILP,

$$x_i \in \{0, 1\}.$$

LP is solvable in polynomial time. General ILP is NP-hard, even with binary variables.

Simple explanation

ILP is a modeling language for “choose discrete decisions under linear rules.” Binary variables are yes/no switches. The hard part is not writing the model; it is that requiring variables to be integer destroys the nice geometry that makes ordinary LP polynomial-time solvable.

Modeling with ILP

Typical modeling choices:

- Binary decision variable: $x_i = 1$ iff item/action/edge/job is selected.
- Assignment variable: $x_{ij} = 1$ iff object i is assigned to option j .
- Ordering variable: $y_{ij} = 1$ iff i precedes j .
- Flow variable: f_e is amount sent through edge e .

Common constraints:

- Select exactly one: $\sum_j x_{ij} = 1$.
- Capacity: $\sum_i a_i x_i \leq C$.
- Logical implication: $x \leq y$ means $x = 1 \Rightarrow y = 1$.
- Either-or via big-M: $a^T x \leq b + M(1 - y)$.

Shortest path ILP formulation

For directed graph $G = (V, E)$ with source s , target t , edge costs c_e , let $x_e \in \{0, 1\}$ indicate whether edge e is selected.

$$\min \sum_{e \in E} c_e x_e$$

subject to flow conservation:

$$\sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = \begin{cases} 1 & v = s, \\ -1 & v = t, \\ 0 & \text{otherwise.} \end{cases}$$

For shortest path, the LP relaxation $0 \leq x_e \leq 1$ already has integral optimal solutions because the node-arc incidence matrix is totally unimodular.

Traveling salesman ILP formulation

For complete graph $G = (V, E)$ with costs c_{ij} , binary variable $x_{ij} = 1$ iff edge $\{i, j\}$ is in the tour.

Degree constraints:

$$\sum_{j \neq i} x_{ij} = 2 \quad \forall i \in V.$$

Subtour elimination constraints:

$$\sum_{\{i,j\} \in E(S)} x_{ij} \leq |S| - 1 \quad \forall S \subset V, 2 \leq |S| \leq |V| - 1.$$

Objective:

$$\min \sum_{\{i,j\} \in E} c_{ij} x_{ij}.$$

This Dantzig-Fulkerson-Johnson formulation has exponentially many subtour constraints, but they can be separated iteratively. Alternative compact formulations include MTZ constraints with order variables, usually weaker but smaller.

Branch and Bound

Branch and Bound is an exact tree-search framework for hard optimization problems.

For minimization:

1. Solve a relaxation of the current subproblem to get lower bound LB .
2. If relaxation is infeasible, prune.
3. If $LB \geq UB$ where UB is best known feasible solution, prune.
4. If relaxation solution is integral/feasible for original problem, update UB .
5. Otherwise branch, e.g. choose fractional variable x_i and create $x_i \leq \lfloor x_i^* \rfloor$ and $x_i \geq \lceil x_i^* \rceil$.

Worst-case complexity remains exponential, but good bounds and branching rules make many practical instances solvable.

Other exact ILP methods

For completeness, the source notes also mention the standard family of exact ILP methods:

- **Enumerative methods:** try feasible integer assignments, often only practical for tiny instances.
- **Cutting planes:** solve the LP relaxation, add valid inequalities that cut off the current fractional solution, and repeat.
- **Branch and cut:** combine branch-and-bound with dynamically generated cutting planes. This is the basis of many modern ILP solvers.

Special ILP problems solvable in polynomial time

Some ILPs have integral LP relaxations. Important sufficient condition: the constraint matrix is **totally unimodular** and the right-hand side is integral.

Examples:

- Shortest path and many network-flow formulations.
- Maximum flow and minimum-cost flow.
- Bipartite matching.
- Assignment problem.
- Certain interval/scheduling matrices.

In these cases, solving the LP gives an integral optimum.

2. Shortest Paths

Problem variants

Given graph $G = (V, E)$ and edge lengths $c : E \rightarrow \mathbb{R}$:

- Single-source shortest paths: distances from s to all vertices.
- Single-pair shortest path: distance from s to t .
- All-pairs shortest paths: distances between all pairs.

If a negative cycle is reachable from s and can reach t , no finite shortest path from s to t exists.

Bellman's principle of optimality: every subpath of a shortest path is itself shortest between its endpoints.

Relaxation

For edge (u, v) , relaxation checks whether going through u improves v :

$$\text{if } d[v] > d[u] + c(u, v), \text{ set } d[v] = d[u] + c(u, v).$$

Store predecessor pointers to reconstruct paths.

Example

If currently $d[A] = 3$ and edge (A, B) has cost 4, then the path to B through A has cost 7. If $d[B] = 10$, relaxation updates $d[B]$ to 7 and stores predecessor A . If $d[B] = 6$, nothing changes.

Dijkstra algorithm

Assumption: all edge lengths are nonnegative.

1. Initialize $d[s] = 0$, all others ∞ .
2. Maintain priority queue keyed by tentative distance.
3. Repeatedly extract vertex u with minimum distance.
4. Relax all outgoing edges (u, v) .

Once a vertex is extracted, its distance is final. Complexity with binary heap: $O((n + m) \log n)$, usually written $O(m \log n)$. With Fibonacci heap: $O(m + n \log n)$.

Applications: routing with nonnegative costs, road networks, uniform-cost search.

Bellman-Ford algorithm

Allows negative edges and detects reachable negative cycles.

1. Initialize distances.
2. Repeat $n - 1$ times: relax every edge.
3. Do one extra pass; if any edge can still be relaxed, a negative cycle is reachable.

Complexity: $O(nm)$.

Applications: difference constraints, graphs with negative penalties, arbitrage detection.

Floyd-Warshall algorithm

All-pairs shortest paths by dynamic programming. Let $D^{(k)}[i, j]$ be the shortest distance from i to j using only intermediate vertices from $\{1, \dots, k\}$.

Recurrence:

$$D^{(k)}[i, j] = \min(D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j]).$$

Implementation:

```
for k in 1..n:
  for i in 1..n:
    for j in 1..n:
      D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

Complexity: $O(n^3)$ time, $O(n^2)$ memory. Negative cycle exists iff some $D[i, i] < 0$ after the algorithm.

Shortest paths in DAGs

In a directed acyclic graph:

1. Topologically sort vertices.
2. Initialize $d[s] = 0$.
3. Process vertices in topological order and relax outgoing edges.

Complexity: $O(n + m)$, works with negative edges because there are no cycles.

Problem formulations using shortest paths

Shortest-path models appear in:

- Difference constraints: $x_v - x_u \leq c(u, v)$.
- Project scheduling with precedence constraints.
- Dynamic programming state graphs.
- Resource-constrained path variants.
- Viterbi-style decoding in layered graphs.

3. Network Flows

Maximum flow

For directed graph $G = (V, E)$, source s , sink t , capacities $u_e \geq 0$, a flow f satisfies:

Capacity:

$$0 \leq f_e \leq u_e.$$

Flow conservation for $v \neq s, t$:

$$\sum_{e \in \delta^-(v)} f_e = \sum_{e \in \delta^+(v)} f_e.$$

Flow value:

$$|f| = \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e.$$

Goal: maximize $|f|$.

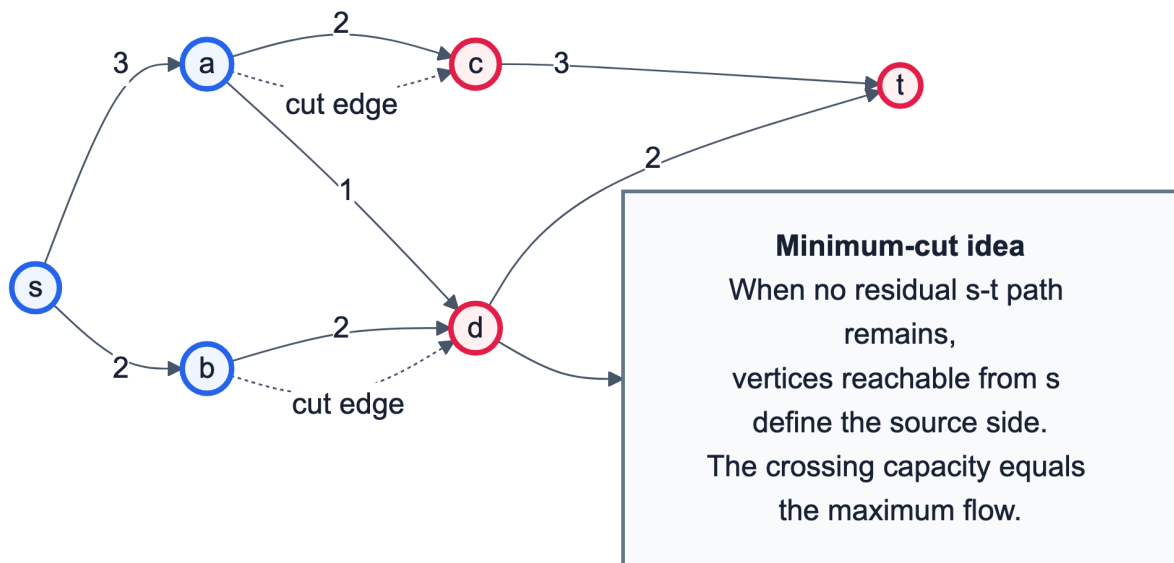


Figure 2: Maximum-flow/minimum-cut picture: augmenting paths improve flow until the residual reachable set defines a minimum cut.

Residual graph

For each edge (u, v) :

- Forward residual capacity: $u(u, v) - f(u, v)$.
- Backward residual capacity: $f(u, v)$.

An augmenting path is an s - t path in the residual graph with positive residual capacity. Its bottleneck is the minimum residual capacity along the path.

Example

If an augmenting path has residual capacities 5, 2, 7, the bottleneck is 2. Increase flow by 2 along every forward edge on the path. Any backward edge means “undo 2 units” of previous flow on that original edge.

Ford-Fulkerson algorithm

1. Start with zero flow.
2. While an augmenting path exists, augment by its bottleneck capacity.
3. Stop when no augmenting path exists.

With integral capacities, terminates with maximum flow. Complexity depends on path choice; with arbitrary real capacities it may not terminate.

Edmonds-Karp chooses shortest augmenting paths by BFS and runs in $O(nm^2)$.

Minimum cut and max-flow min-cut theorem

An s - t cut is a partition $(S, V \setminus S)$ with $s \in S$, $t \notin S$. Its capacity is

$$c(S, V \setminus S) = \sum_{u \in S, v \notin S} u(u, v).$$

Max-flow min-cut theorem:

$$\max |f| = \min c(S, V \setminus S).$$

At the end of Ford-Fulkerson, vertices reachable from s in the residual graph define a minimum cut.

Simple explanation

The residual graph shows where more flow can still be pushed or undone. When the sink is no longer reachable in this graph, the reachable vertices form the source side of a bottleneck cut. That is why “no augmenting path” proves optimality.

Feasible flow with balances

General circulation with balances has node demands/supplies $b(v)$:

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = b(v).$$

Convention: $b(v) > 0$ means demand, $b(v) < 0$ means supply. Feasibility requires $\sum_v b(v) = 0$.

With lower and upper bounds $l_e \leq f_e \leq u_e$:

1. Substitute $f'_e = f_e - l_e$ so $0 \leq f'_e \leq u_e - l_e$.
2. Adjust balances:

$$b'(v) = b(v) - \sum_{e \in \delta^-(v)} l_e + \sum_{e \in \delta^+(v)} l_e.$$

3. Add super-source to supply nodes and super-sink from demand nodes, or use equivalent circulation construction.
4. Feasible iff all artificial demand/supply edges can be saturated.

Minimum-cost flow

Given capacities, balances, and costs c_e , minimize

$$\sum_{e \in E} c_e f_e$$

subject to flow constraints.

Minimum-cost flow generalizes shortest path, assignment, transportation, and some scheduling problems.

Cycle-canceling algorithm

1. Start with any feasible flow.
2. Build residual graph with residual costs: forward edge cost c_e , backward edge cost $-c_e$.
3. If the residual graph contains a negative-cost cycle, augment flow around it as much as possible.
4. Repeat until no negative cycle remains.

Optimality condition: a feasible flow is minimum-cost iff its residual graph has no negative-cost cycle.

Maximum cardinality matching via flow

For bipartite graph $G = (U \cup W, E)$:

1. Add source s and sink t .
2. Add edges $s \rightarrow u$ for $u \in U$, capacity 1.
3. Direct original edges $u \rightarrow w$ with capacity 1.
4. Add edges $w \rightarrow t$ for $w \in W$, capacity 1.

Maximum integral flow corresponds to maximum cardinality matching. Integrality follows from integral capacities.

Problem formulations using flows

Flows model:

- Assignment and bipartite matching.
- Transportation and logistics.
- Image segmentation by min cut.
- Project selection.
- Disjoint paths and connectivity.
- Scheduling with time-expanded networks.

4. Knapsack

0/1 knapsack

Input: items $i = 1, \dots, n$ with weight $w_i > 0$, profit $p_i > 0$, capacity W .

$$\max \sum_i p_i x_i$$

subject to:

$$\sum_i w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

0/1 knapsack is weakly NP-hard.

Fractional knapsack

If fractions are allowed, greedy by decreasing ratio p_i/w_i is optimal. Complexity is dominated by sorting: $O(n \log n)$.

Dynamic programming by weight

Let $DP[i][w]$ be maximum profit using first i items with capacity w .

$$DP[i][w] = \begin{cases} DP[i-1][w], & w_i > w, \\ \max(DP[i-1][w], DP[i-1][w-w_i] + p_i), & w_i \leq w. \end{cases}$$

Complexity: $O(nW)$ time and $O(W)$ memory with rolling array. This is pseudo-polynomial, not polynomial in input bit-length.

Dynamic programming by profit

Let $DP[i][p]$ be minimum weight needed to achieve profit exactly p using first i items. Minimize weight and find largest profit with weight $\leq W$.

Complexity: $O(nP)$ where $P = \sum_i p_i$.

Approximation and FPTAS

A simple 2-approximation:

1. Take greedy items by profit/weight ratio until next item does not fit.
2. Compare this set with the single most profitable item that fits.
3. Return the better one.

This achieves at least $OPT/2$.

FPTAS by profit scaling:

1. Choose scaling factor $K = \epsilon p_{\max}/n$.
2. Replace profits by $p'_i = \lfloor p_i/K \rfloor$.
3. Run profit-DP on scaled profits.

Runs polynomially in n and $1/\epsilon$, and returns a $(1 - \epsilon)$ -approximation for maximization.

5. Traveling Salesman Problem

Problem statement and complexity

TSP asks for a minimum-cost Hamiltonian cycle visiting every city exactly once.

Decision version: is there a tour of length at most B ?

TSP is NP-hard. The decision version is NP-complete. General metric-free TSP has no finite approximation ratio unless $P=NP$, because missing Hamiltonian edges can be encoded as very expensive edges.

Metric TSP

Metric TSP assumes:

$$c(i, j) \geq 0, \quad c(i, j) = c(j, i), \quad c(i, j) \leq c(i, k) + c(k, j).$$

The triangle inequality enables shortcutting repeated vertices without increasing tour length.

Double-tree algorithm

1. Compute an MST T .
2. Double every edge of T ; the resulting multigraph is Eulerian.
3. Traverse an Euler tour.
4. Shortcut already visited vertices to obtain a Hamiltonian cycle.

Guarantee:

$$w(T) \leq OPT,$$

so doubled tree has weight $2w(T) \leq 2OPT$, and shortcutting does not increase weight. Approximation ratio: 2.

Christofides algorithm

1. Compute MST T .
2. Let O be the set of odd-degree vertices in T .
3. Compute a minimum-weight perfect matching M on O .
4. Add M to T ; all degrees become even.
5. Find Euler tour and shortcut repeated vertices.

Guarantee:

$$w(T) \leq OPT, \quad w(M) \leq \frac{1}{2}OPT.$$

Therefore total weight is at most $\frac{3}{2}OPT$. Approximation ratio: 1.5 for metric TSP.

Exam tip

Both double-tree and Christofides rely on the triangle inequality. Without it, shortcutting repeated vertices can make the tour more expensive, and the approximation proof collapses.

Local search and k-OPT

k-OPT improves a tour by replacing k edges with k different edges, preserving a Hamiltonian cycle.

- 2-OPT removes two crossing or bad edges and reconnects.
- 3-OPT is stronger but more expensive.
- General k-OPT grows quickly with k .

Local search stops at a local optimum: no improving k-edge exchange exists. It is usually heuristic; quality can be good in practice, but global optimality is not guaranteed.

6. Scheduling

Basic definitions

Input consists of jobs $J_1, \dots, J_n$ and resources/machines. Common job parameters:

- p_j : processing time.
- r_j : release time, earliest start.
- $\tilde{d}_j$: hard deadline.
- d_j : due date used in lateness/tardiness objectives.
- w_j : job weight.
- C_j : completion time.
- $L_j = C_j - d_j$: lateness.
- $T_j = \max(0, C_j - d_j)$: tardiness.
- $C_{\max} = \max_j C_j$: makespan.

Preemption (**pmtn**) means a job can be interrupted and resumed later.

Graham notation

Scheduling problems are denoted:

$$\alpha \mid \beta \mid \gamma.$$

- α : machine environment. Examples: 1 one machine, P identical parallel machines, Q uniform parallel machines, R unrelated machines.
- β : job constraints. Examples: **r_j**, **d_j**, **pmtn**, **prec**.
- γ : objective. Examples: $C_{\max}$, $\sum C_j$, $\sum w_j C_j$, $L_{\max}$, $\sum T_j$.

Simple explanation

Read Graham notation as “machine environment | job restrictions | objective.” For example, $P|prec|C_{\max}$ means identical parallel machines, precedence constraints between jobs, and the goal is to minimize the finish time of the last job.

One resource: Bratley algorithm

Bratley’s algorithm is a branch-and-bound algorithm for single-machine scheduling with release times and deadlines, often written:

$$1 \mid r_j, \tilde{d}_j \mid C_{\max}$$

as a feasibility/makespan problem.

It searches over job orders. At a partial schedule:

- Current time t is the completion time of scheduled jobs.
- A candidate job j starts at $\max(t, r_j)$ and completes at $C_j = \max(t, r_j) + p_j$.
- If $C_j > \tilde{d}_j$, that branch is infeasible.

Pruning rules:

- **Deadline pruning**: if any remaining job cannot meet its deadline even when scheduled next, prune.
- **Bound pruning**: if a lower bound on final makespan exceeds best known solution, prune.

- **Dominance/decomposition:** if no remaining job is released before current idle boundary, some ordering choices become irrelevant.

Worst-case exponential, but effective for moderate instances.

One resource: Horn algorithm

Horn's algorithm solves the preemptive single-machine problem:

$$1 \mid r_j, \text{pmtn} \mid L_{\max}.$$

Rule: at every time, among available jobs, process the job with earliest due date/deadline. This is the preemptive earliest-deadline-first rule.

Implementation with priority queue by due date gives $O(n \log n)$ after sorting events. It is optimal for minimizing maximum lateness with release times and preemption.

Parallel identical resources: list scheduling

For identical parallel machines and precedence constraints:

$$P \mid \text{prec} \mid C_{\max},$$

list scheduling keeps a priority list of available jobs whose predecessors are completed. Whenever a machine becomes free, assign the next available job from the list.

For m identical machines, Graham's list scheduling has approximation guarantee:

$$C_{\max}^{LS} \leq \left(2 - \frac{1}{m}\right) OPT.$$

If jobs are independent and sorted by longest processing time first (LPT), the guarantee improves to:

$$\frac{4}{3} - \frac{1}{3m}.$$

Parallel identical resources: McNaughton algorithm

For the preemptive identical-machine problem

$$P \mid \text{pmtn} \mid C_{\max},$$

McNaughton's algorithm gives an optimal schedule in $O(n)$ time after the jobs are ordered. The optimal makespan is

$$C_{\max}^* = \max \left\{ \max_j p_j, \frac{1}{m} \sum_j p_j \right\}.$$

Place the jobs one after another into an $m \times C_{\max}^*$ rectangle. When a job crosses a machine boundary, split it there; preemption makes this legal. The lower bound is therefore attainable.

Parallel identical resources: dynamic programming

For fixed number of machines m , problem

$$P_m \parallel C_{\max}$$

can be solved pseudo-polynomially by dynamic programming over machine loads. For two machines, this is closely related to PARTITION.

For constant m , maintain reachable load vectors after scheduling first i jobs. Complexity is pseudo-polynomial in total processing time and exponential in m .

Project scheduling with temporal constraints

Project scheduling has activities and temporal relations such as:

$$s_j - s_i \geq a_{ij}$$

or equivalently difference constraints. If only temporal constraints exist and resources are unlimited, feasibility and earliest times can be solved by longest paths in a constraint graph, or by Bellman-Ford after sign conversion.

With renewable resource constraints, project scheduling becomes NP-hard.

Relative-order ILP formulation Variables:

- s_i : start time of job i .
- $y_{ij} = 1$ iff job i is before job j on a shared resource.

For jobs i, j needing the same unary resource:

$$s_i + p_i \leq s_j + M(1 - y_{ij}),$$

$$s_j + p_j \leq s_i + My_{ij}.$$

Temporal constraints are linear:

$$s_j - s_i \geq a_{ij}.$$

Objective may minimize $C_{\max}$ with constraints $C_{\max} \geq s_i + p_i$.

Time-indexed ILP formulation For discrete time horizon T , let $x_{jt} = 1$ iff job j starts at time t .

Each job starts once:

$$\sum_{t=0}^{T-p_j} x_{jt} = 1.$$

Resource capacity R at each time τ :

$$\sum_j \sum_{t: t \leq \tau < t+p_j} r_{jk} x_{jt} \leq R_k.$$

Time-indexed formulations can be large but often have strong relaxations.

Example

If job j has processing time $p_j = 3$ and starts at $t = 5$, then it occupies times 5, 6, 7. In a time-indexed ILP, capacity constraints for time slots 5, 6, and 7 all include the variable $x_{j,5}$.

7. Constraint Satisfaction Problem and AC3

CSP definition

A constraint satisfaction problem consists of:

- Variables $X_1, \dots, X_n$.
- Domains D_i of possible values.
- Constraints restricting allowed combinations of values.

A solution assigns every variable a value from its domain satisfying all constraints.

Examples: graph coloring, Sudoku, scheduling, map coloring, configuration problems.

General CSP is NP-complete, but propagation and structure can make many instances easy.

Arc consistency

For binary constraint C_{ij} between variables X_i and X_j , arc (i, j) is arc-consistent iff for every value $a \in D_i$ there exists value $b \in D_j$ such that (a, b) satisfies C_{ij} .

If no such b exists, a can be removed from D_i .

AC3 algorithm

AC3 enforces arc consistency.

AC3:

```
queue = all directed arcs (Xi, Xj)
while queue not empty:
    (Xi, Xj) = pop(queue)
    if revise(Xi, Xj):
        if Di is empty: return inconsistent
        for each neighbor Xk of Xi, k != j:
            push(queue, (Xk, Xi))
return arc-consistent
```

```
revise(Xi, Xj):
    changed = false
    for each a in Di:
        if no b in Dj satisfies Cij(a,b):
            remove a from Di
            changed = true
    return changed
```

For n variables, maximum domain size d , and e binary constraints, a standard bound is $O(ed^3)$; with optimized support tracking it can be improved.

Arc consistency is not complete in general: after AC3, search/backtracking may still be needed.

Exam Checklist

Be able to:

- Formulate shortest path and TSP as ILPs.
- Explain Branch and Bound and LP relaxation bounds.
- Name ILP cases where LP relaxation is integral.
- State Dijkstra, Bellman-Ford, Floyd-Warshall, and DAG shortest paths with assumptions and complexities.
- Define residual graph, augmenting path, max-flow min-cut theorem, and Ford-Fulkerson.
- Transform feasible flow with lower bounds and balances.
- Explain minimum-cost flow and negative-cycle optimality.
- Reduce bipartite matching to max flow.
- Give knapsack DP, pseudo-polynomial meaning, 2-approximation, and FPTAS.
- Prove double-tree 2-approximation and Christofides $3/2$ -approximation for metric TSP.
- Explain Graham scheduling notation, Bratley, Horn, list scheduling, McNaughton, and DP for fixed parallel machines.
- Build relative-order and time-indexed ILP formulations for project scheduling.
- Define CSP, arc consistency, and AC3.